

DSA STUDY BLUEPRINT SERIES

96. Lowest Common Ancestor in BST

Finding Lowest Common Ancestor in BSTs

Section 09 • C++ Core Data Structures & Algorithms

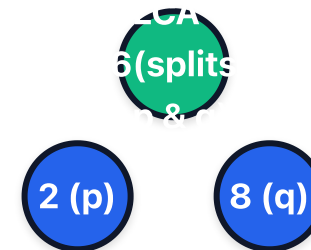
Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **BST LCA:** Can be resolved without traversing the entire tree.
- LCA found if current node value lies between target inputs.

Memory & Execution Blueprint



C++ Implementation Reference

Standard code layout and syntax

```
TreeNode* lowestCommonAncestorBST(TreeNode* root, TreeNode* p, TreeNode* q) {  
    if (root->val > p->val && root->val > q->val)  
        return lowestCommonAncestorBST(root->left, p, q);  
    if (root->val < p->val && root->val < q->val)  
        return lowestCommonAncestorBST(root->right, p, q);  
    return root;  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(H)$
Space Complexity	$O(H)$

Key Practices & Gotchas

- **Important:** Runs in $O(H)$ time, which is faster than general binary tree search.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace BST constraint validation checks verifying node ranges `[Min, Max]`:

Active Node	Valid range limit	Comparison check	Recursive branch step
15 (Root)	$[-\infty, +\infty]$	True $(-\infty < 15 < +\infty)$	Recurse Left child with range $[-\infty, 15]$
10 (Left)	$[-\infty, 15]$	True $(-\infty < 10 < 15)$	Recurse Right child with range $[10, 15]$

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Inorder property:** BST Inorder traversals yield strictly increasing sorted lists.
- **Tree Balancing:** Balanced tree heights (AVL/Red-black) guarantee logarithmic search times.
- **Related LeetCode Problems:** #98 Validate Binary Search Tree, #700 Search in a Binary Search Tree, #108 Convert Sorted Array to Binary Search Tree.